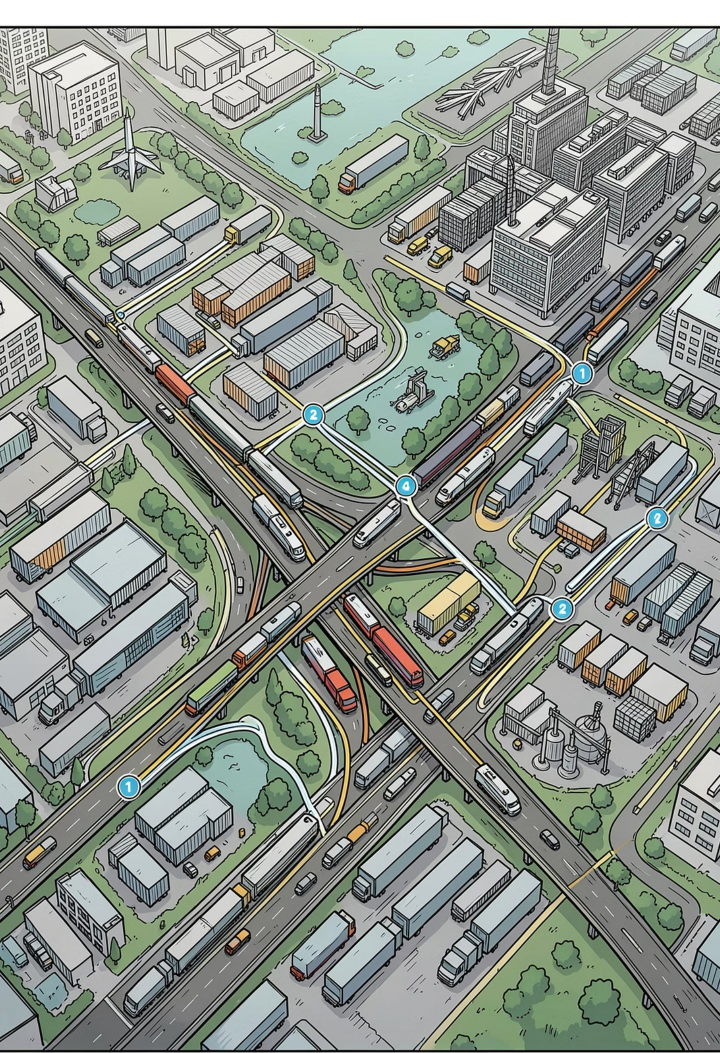




Жизненный цикл разработки программы

Программа предназначена для решения **сбалансированной транспортной задачи методом северо-западного угла**. На первом этапе были определены основные требования: пользователь вводит количество поставщиков и потребителей, запасы, потребности и тарифы перевозки. Программа проверяет сбалансированность задачи ($\text{сумма запасов} = \text{сумма потребностей}$), строит начальный план перевозок и рассчитывает общую стоимость. Основные заинтересованные лица: пользователь программы, преподаватель, проверяющий проект, и разработчик программы.

УЧЕБНЫЙ ПРОЕКТ

МЕТОДЫ ОПТИМИЗАЦИИ

Классификация требований

Требования к программе делятся на **функциональные** (что программа должна делать) и **нефункциональные** (как программа должна работать). Такое разделение позволяет структурировать разработку и чётко определить критерии приёмки.

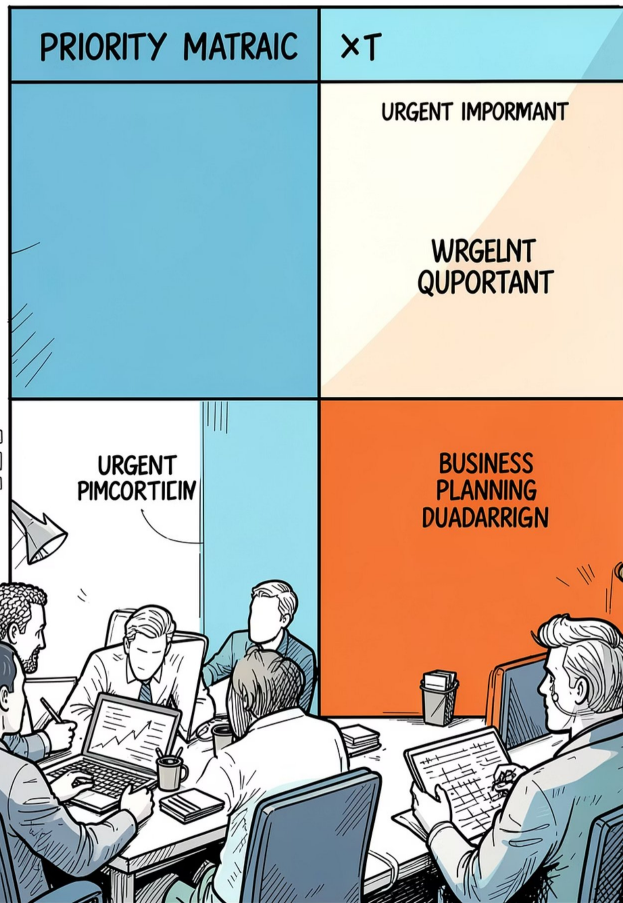
Функциональные требования

- Ввод количества поставщиков и потребителей
- Ввод запасов поставщиков и потребностей потребителей
- Ввод тарифов перевозки между парами
- Проверка баланса транспортной задачи
- Построение плана методом северо-западного угла
- Вывод плана перевозок и расчёт общей стоимости
- Возможность повторного решения без перезапуска

Нефункциональные требования

- Работа в консольном режиме
- Простой и понятный интерфейс
- Проверка корректности ввода чисел
- Запрет отрицательных значений
- Сообщения об ошибках при неверном вводе
- Реализация на языке C#
- Разделение кода на отдельные методы

PRIORITY MACRRIU



Приоритизация требований

На этапе приоритизации были выделены ключевые функции программы. Это позволило сосредоточить усилия на реализации самого важного и обеспечить работоспособность программы даже при ограниченных ресурсах.

Обязательные требования

Без этих функций программа не может выполнять своё назначение. Они реализуются в первую очередь и являются минимально необходимым набором для рабочей версии.

- Ввод исходных данных
- Проверка сбалансированности задачи
- Построение плана перевозок
- Расчёт общей стоимости
- Вывод результата пользователю

Дополнительные требования

Эти функции повышают удобство работы с программой, но не являются критичными для базовой функциональности. Реализуются после обязательных требований.

- Повторное решение задачи без перезапуска
- Безопасный ввод чисел (метод ReadInt)
- Защита от ввода текста и отрицательных чисел

Документирование требований

После определения требований была составлена **спецификация программы** — документ, фиксирующий назначение, входные и выходные данные, а также ограничения системы. Спецификация служит основой для разработки и проверки соответствия результата ожиданиям.

Название	Программа решения сбалансированной транспортной задачи методом северо-западного угла
Назначение	Построение начального плана перевозок в сбалансированной транспортной задаче и вычисление общей стоимости перевозок
Входные данные	Количество поставщиков и потребителей, запасы поставщиков, потребности потребителей, тарифы перевозки между каждой парой
Выходные данные	Сообщение о сбалансированности задачи, таблица плана перевозок, общая стоимость перевозок
Ограничения	Только сбалансированные задачи; неотрицательные целые числа; работа через консоль

Системное проектирование

На этапе системного проектирования программа была разделена на **шесть логических модулей**. Каждый модуль отвечает за отдельный аспект работы программы, что упрощает разработку, тестирование и поддержку кода.

01

Модуль ввода данных

Получает от пользователя количество поставщиков, потребителей, запасы, потребности и тарифы.

02

Модуль проверки баланса

Сравнивает сумму запасов и сумму потребностей. При несовпадении расчёт не выполняется.

03

Модуль построения плана

Реализует метод северо-западного угла: распределение начинается с верхней левой ячейки и движется вправо или вниз.

04

Модуль расчёта стоимости

Умножает количество груза по каждому маршруту на тариф и суммирует все значения.

05

Модуль вывода результата

Отображает пользователю готовый план перевозок и итоговую стоимость.

06

Модуль повторного запуска

Позволяет пользователю решить новую задачу после завершения предыдущей без перезапуска программы.

Детальное проектирование

На этапе детального проектирования была определена **внутренняя структура программы**: переменные, массивы и методы. Чёткое разделение ответственности между методами делает код понятным и легко поддерживаемым.

Основные переменные

- `m` — количество поставщиков
- `n` — количество потребителей
- `supply` — массив запасов поставщиков
- `demand` — массив потребностей потребителей
- `cost` — двумерный массив тарифов
- `plan` — двумерный массив плана перевозок
- `totalSupply`, `totalDemand` — суммы запасов и потребностей
- `totalCost` — общая стоимость перевозок

Основные методы

- `Main` — главный метод, управляет основным сценарием работы программы
- `NorthWestCorner` — реализует метод северо-западного угла, возвращает таблицу перевозок
- `Sum` — считает сумму элементов массива, используется для проверки баланса
- `ReadInt` — обеспечивает безопасный ввод целых неотрицательных чисел



Каждый метод решает одну задачу — это принцип единой ответственности (Single Responsibility Principle).

Проектирование пользовательского интерфейса

Программа имеет **консольный интерфейс** — пользователь взаимодействует с ней через текстовые сообщения. Интерфейс построен последовательно: программа пошагово запрашивает данные, затем показывает результат или сообщение об ошибке.

Структура диалога

1. Ввод количества поставщиков и потребителей
2. Ввод запасов поставщиков
3. Ввод потребностей потребителей
4. Ввод тарифов перевозки
5. Отображение сумм запасов и потребностей
6. Вывод плана перевозок и стоимости (или ошибки)

Пример работы интерфейса

Количество поставщиков: 2

Количество потребителей: 3

Запасы поставщиков:

Поставщик 1: 20

Поставщик 2: 30

Потребности потребителей:

Потребитель 1: 10

Потребитель 2: 25

Потребитель 3: 15

Консольный интерфейс выбран потому, что он прост, не требует графических библиотек и идеально подходит для учебного проекта.

Проектирование данных

Для хранения данных транспортной задачи используются **одномерные и двумерные массивы**. Выбор структур данных обусловлен табличной природой транспортной задачи: поставщики и потребители естественно представляются в виде матрицы.



Массив supply

Одномерный массив. Каждый элемент хранит запас одного поставщика. Длина массива равна m (количество поставщиков).



Массив demand

Одномерный массив. Каждый элемент хранит потребность одного потребителя. Длина массива равна n (количество потребителей).



Массив cost

Двумерный массив тарифов. Строки соответствуют поставщикам, столбцы — потребителям. Размер: $m \times n$.



Массив plan

Двумерный массив результата. Содержит количество груза, которое нужно перевезти от каждого поставщика к каждому потребителю.

Настройка среды разработки

Для разработки программы была использована среда **Visual Studio** — интегрированная среда разработки (IDE) от Microsoft. Она позволяет писать, запускать и отлаживать код в одном месте, что особенно удобно для учебных проектов на C#.

Создание проекта

- 1 Открыть Visual Studio → Создать проект → Консольное приложение → C# → Указать имя проекта

Написание кода

- 2 Открыть файл Program.cs и вставить программный код

Запуск программы

- 3 Нажать кнопку «Пуск» или клавишу F5 для запуска и проверки работы



Написание кода и алгоритм метода северо-западного угла

Программный код написан на языке **C#**. В методе `Main` выполняется основная логика: ввод данных, проверка баланса, построение плана и вывод результата. Ключевой метод `NorthWestCorner` реализует следующий алгоритм:

